Technical Report

Name:Zhu Yunhao

Student id:201690984

Link to GitHub repository: https://github.com/planeboom/XJCO3011

Link to API documentation:
https://github.com/planeboom/XJCO3011/blob/main/API_Documentation.pdf

Link to your presentation slides:

https://github.com/planeboom/XJCO3011/blob/main/pre.pptx

1. Introduction

This project provides a rest-style web API to manage and query the nutritional data of Starbucks beverages. The system provides a standardized JSON interface for client APP applications that strictly abide by REST mode constraints.

The data source and preprocessing API are supported by the actual Kaggle data set containing complex nutritional indicators such as calories, fat and sugar. Because the data in the real world is essentially unstructured, it is necessary to develop a powerful data cleaning script to analyze the original CSV file, safely handle the lost variables and map them to the relational database at the critical initial stage.

Key Achievements & Deliverables To meet and exceed coursework specifications, this web service successfully delivers:

- Complete CRUD Functionality: Standardized implementation of GET, POST, PUT, PATCH, and DELETE operations, featuring a clear programmatic distinction between full resource replacement (PUT) and partial modification (PATCH).
- Live Cloud Deployment: The backend system has been successfully migrated from a local environment and is actively hosted on an external web server (PythonAnywhere).
- Interactive Documentation: Comprehensive API specifications were validated using Postman and dynamically rendered via ReDoc, providing clear endpoint definitions and request/response examples.

2. System Architecture and Technology Stack

2.1 System Architecture & Data Flow

This project uses a standard three-tier client/server architecture to clearly isolate the concerns among data storage, business logic and presentation.

As shown in Figure 1, the client's HTTP request is received by PythonAnywhere WSGI server and routed through Django URL configuration. ViewSet handles the core business logic and uses DRF serializer to convert the instances of complex models into native Python data types and draw them as JSON payloads. The Django Object-Relational Mapper (ORM) safely transforms Python commands into SQL queries and interacts with the database.

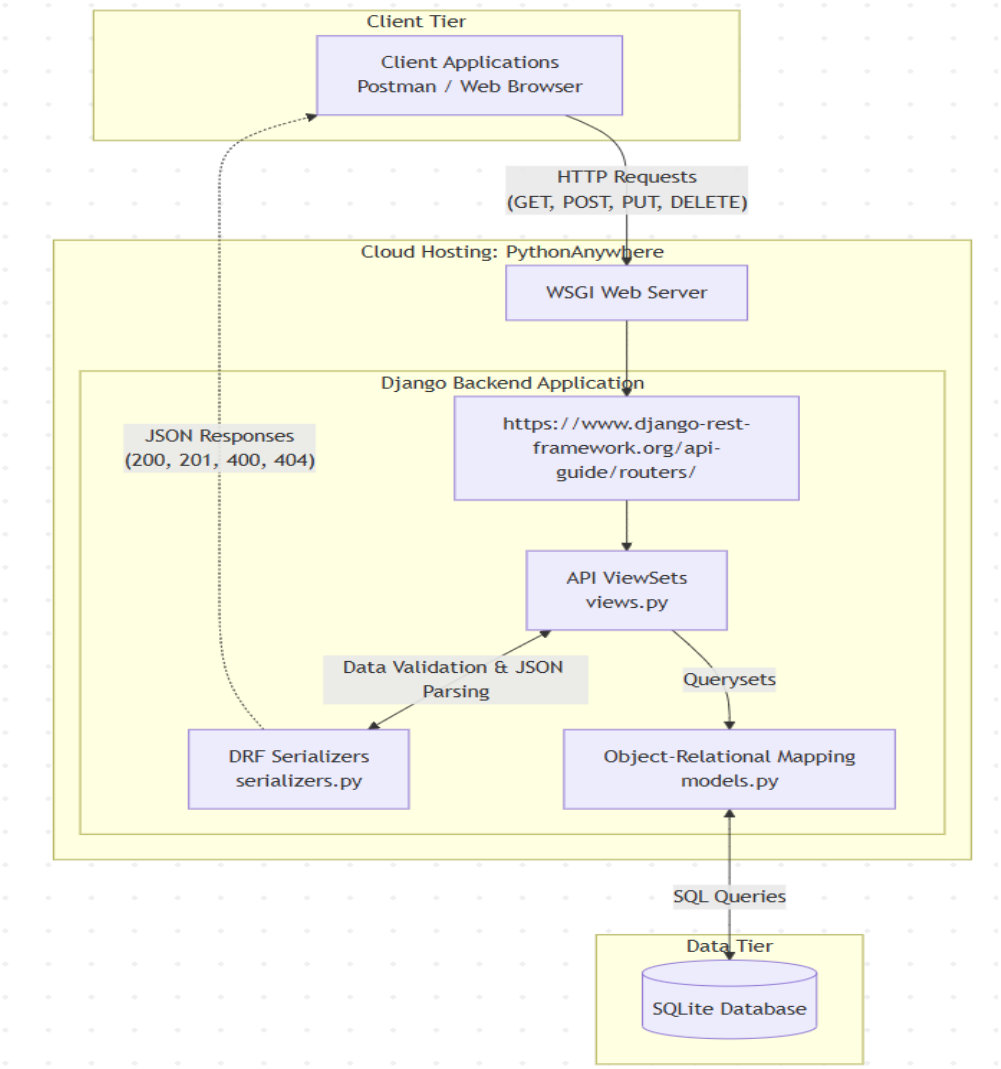

Figure 1: System Architecture and Data Flow

2.2 Technology Stack Justification

To ensure a maintainable and standard-compliant web service, the following technologies were selected:

- **Framework: Django & Django REST Framework**
  Used for its robust built-in ORM and efficient serialization, which ensures fast development and standardized JSON output.
- **Database: SQLite**
  A lightweight, file-based relational database that is perfectly suited for the

scale of this project.

- **Hosting: PythonAnywhere**
  A reliable Platform-as-a-Service (PaaS) used to host the API in a live cloud environment.

3. API Design and Implementation

The API was designed strictly following RESTful principles, utilizing predictable resource-oriented URLs and leveraging standard HTTP methods to manipulate the Beverage entities.

3.1 RESTful Endpoints and CRUD Operations

Instead of hardcoding individual routing functions, the project utilizes DRF's ModelViewSet and DefaultRouter. This approach automatically generates a comprehensive set of RESTful endpoints under the base route /api/beverages/, seamlessly mapping HTTP verbs to CRUD operations:

- Create (POST): POST /api/beverages/ allows the creation of a new beverage record. The API rigorously validates the incoming JSON payload against the BeverageSerializer to ensure data integrity (e.g., verifying that numerical fields like calories are valid).
- Read (GET): GET /api/beverages/ retrieves a complete JSON array of all menu items, while GET /api/beverages/{id}/ fetches the specific nutritional detail of a single beverage using its primary key.
- Update (PUT & PATCH): The API explicitly supports two distinct update mechanisms. PUT /api/beverages/{id}/ is implemented for full replacement, mandating a complete JSON representation of the beverage (missing required fields will return a 400 Bad Request). Conversely, PATCH /api/beverages/{id}/ allows for partial modifications (e.g., exclusively lowering the calories value), minimizing network bandwidth and providing flexibility for client-side applications.
- Delete (DELETE): DELETE /api/beverages/{id}/ removes a designated resource from the database, returning a standard 204 No Content status upon successful deletion.

3.2 Data Validation and HTTP Status Codes

To ensure a reliable developer experience for API consumers, the system is configured to return semantically accurate HTTP status codes:

- 200 OK for successful queries and updates.
- 201 Created specifically upon the successful execution of a POST request.
- 400 Bad Request when client-supplied JSON data fails validation (e.g., missing mandatory fields or incorrect data types).

- 404 Not Found when a client attempts to access or modify an {id} that does not exist in the database.

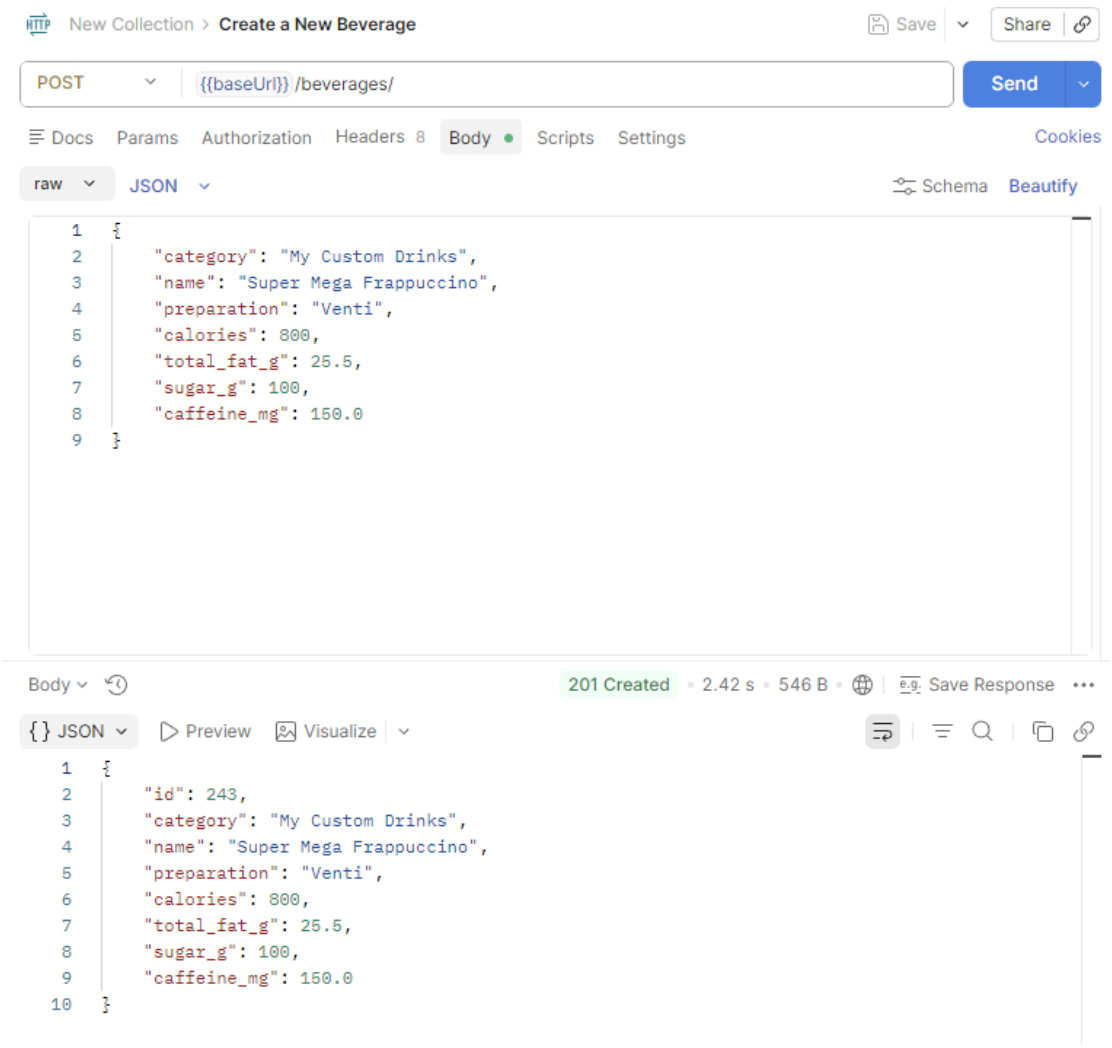

Figure 2: Demonstration of a successful POST request and JSON response via Postman

4. Deployment and Testing

To ensure the API's reliability and to meet the advanced requirements of this coursework, the project underwent rigorous API testing and was subsequently deployed to a live production environment.

4.1 API Testing and Documentation Generation

API endpoints were systematically tested using **Postman**. Rather than testing isolated URLs, a complete Postman Collection was engineered to simulate a full client-server interaction lifecycle.

- **Environment Variables:** A global variable ({{baseUrl}}) was implemented. This allowed seamless toggling between the local development server (127.0.0.1:8000) and the live production environment without rewriting individual requests.

- **Documentation:** Upon successful validation of all CRUD operations, the response payloads were saved as examples. This Postman Collection was then utilized to automatically generate the comprehensive, interactive API_Documentation.pdf attached to the GitHub repository.

4.2 Cloud Deployment

To make the API globally accessible, the project was deployed to PythonAnywhere. The deployment process involved:

- **WSGI & Environment Setup**: Configuring the wsgi.py file to connect the web server to the Django application and setting up a dedicated virtual environment.
- **Static Files Management**: To ensure the interactive API documentation (ReDoc/Swagger) renders correctly in production, STATIC_ROOT was configured and static file routing (/static/) was explicitly mapped in the cloud dashboard.

5. Declaration of Generative AI Usage

In accordance with university guidelines, I declare the use of Generative AI as an assistive tool (acting as a debugging assistant and syntax reference) during this coursework. The core architecture, business logic, and final execution remain entirely my original work.

5.1 Assisted Tasks (Where AI was used):

- **Initial Research:** Surveying Kaggle for suitable nutritional datasets and data processing strategies.
- **Scripting Assistance:** Assisting with the try-except boilerplate in import_data.py to safely parse irregular CSV data.
- **Debugging:** Analyzing specific framework-level error logs (e.g., Django 500 Internal Server Error) during the PythonAnywhere deployment to identify static file misconfigurations.

5.2 Original Work (Where AI was NOT used):

- **Architecture & API Design:** The selection of Django/DRF, database modeling (models.py), and the design of RESTful endpoints were independent decisions.
- **Deployment & Testing:** The manual configuration of the PythonAnywhere server (including WSGI setup) and the creation of the Postman testing suites with environment variables were executed entirely by me.

Appendix: Examples of Generative AI Conversation Logs

Example 1: Initial Research & Dataset Selection

I have a university coursework to build a RESTful Web API using Django. I need to find a real-world dataset. Can you suggest some interesting datasets from Kaggle that have a mix of numerical and categorical data suitable for CRUD operations? A food, nutrition, or...

显示思路

Building a food or nutrition API is a brilliant choice for your coursework! Food datasets naturally have a clean mix of categorical data (like food types or dietary labels) and numerical data (like calories, macros, or prices), making them perfect for building RESTful CRUD endpoints with Django.

Here are three excellent datasets from Kaggle that perfectly fit your requirements:

1. Starbucks Menu Nutrition Dataset

- **The Data:** Contains the nutritional information for Starbucks' food and drink menu items.
- **Categorical Data:** Beverage Category (e.g., Classic Espresso Drinks, Smoothies), Beverage Name, Preparation Method (e.g., Short, Venti, Soymilk).
- **Numerical Data:** Calories, Total Fat (g), Sodium (mg), Sugars (g), Caffeine (mg).
- **Why it's great for an API:** It's a highly recognizable, well-structured dataset. You can easily build endpoints that filter numerical data (e.g., `GET /api/beverages?max_calories=200`) or group categorical data (e.g., `GET /api/beverages?category=espresso`). It also leaves plenty of room for `POST` and `PUT` operations, like letting a user add a secret menu item.

2. Global Food & Nutrition Database

- **The Data:** A massive, comprehensive database combining USDA foods and branded products.
- **Categorical Data:** Food Name, Food Type/Category, Allergen Flags (Gluten-free, Dairy-free), Nutri-Score grades.
- **Numerical Data:** Energy per 100g, Macronutrients (Protein, Fat, Carbs), Health Scores.

Example 2: Scripting Assistance (Data Import)

Write a Django import_data.py script using csv.DictReader that uses try-except to safely parse 'calories' and 'caffeine' from starbucks.csv, handling non-numeric values like 'Varies' without crashing.

显示思路

Here is a robust Django standalone script tailored exactly to your requirements. It uses `csv.DictReader` and nested `try-except` blocks to act as a "shield," ensuring that dirty data like "Varies" or empty strings won't crash your database migration.

You can place this script in the root directory of your Django project (the same folder as `manage.py`).

Python

```python
import os
import django
import csv

# 1. Setup Django Environment (Crucial for standalone scripts)
# Replace 'your_project_name' with the actual name of your Django project folder
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'your_project_name.settings')
django.setup()

# 2. Import your models AFTER setting up Django
# Replace 'api.models' with your actual app name
from api.models import Beverage

def import_starbucks_data():
    file_path = 'starbucks.csv'

    # Check if file exists before trying to open it
    if not os.path.exists(file_path):
        print(f"Error: Could not find {file_path}. Please ensure it is in the sar
        return

    # Using utf-8-sig to automatically handle any BOM (Byte Order Mark) at the st
    with open(file_path, mode='r', encoding='utf-8-sig') as file:
        reader = csv.DictReader(file)
```

Example 3: Debugging Cloud Deployment

I just deployed my Django REST Framework project on PythonAnywhere. The API endpoints are working and returning JSON correctly, but my ReDoc and Swagger UI documentation pages are completely unstyled. The CSS is missing, and my browser console shows a 404 Not...

显示思路

This is a classic deployment rite of passage! What you're experiencing happens because Django's built-in development server automatically serves static files when `DEBUG = True`. However, in a production environment (like PythonAnywhere), the web server needs to be explicitly told how to handle and serve those files. Since Django REST Framework relies on static CSS and JavaScript files to render ReDoc and Swagger UI, the pages look completely broken when those files return a 404.

Here is the exact step-by-step process to fix this in both your code and your PythonAnywhere dashboard.

Step 1: Update your `settings.py`

You need to tell Django where to gather all your static files (including the ones hidden inside the Django REST Framework package) so the production server can find them in one place.

Open your `settings.py` and ensure your static file settings look like this:

Python

```python
import os
from pathlib import Path

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent

# ... (other settings) ...

# The URL prefix for static files
STATIC_URL = '/static/'
```